

AUTOMATED 3-TIER INFRASTRUCTURE

Cloud Deployment with Terraform and Ansible



CONTRIBUTORS

Aesha Abdelwahid, Tasneem Adel, Roumaysaa Samy,
Yasmeen Mohamed

UNDER SUPERVISION OF

Eng. Azza Khalil

Table of Contents:

1. Introduction.....	2
2. Overview.....	2
PART 1	3
3. System Architecture design (By Terraform)	3
3.1. Networking Layer (The Infrastructure Foundation)	3
3.1.2 Connectivity & Routing Logic.....	4
3.1.3 Load Balancing Architecture (The Traffic Gateways).....	5
3.1.4. Security Group Chaining	6
3.2. The Compute Layer: Application & Logic Tier.....	9
3.2.1 Dual-Tier EC2 Architecture	9
3.2.2 High Availability with Auto Scaling Groups (ASG).....	10
3.2.3 Dynamic Scaling & Resource Management	10
3.2.4 Integration with Ansible.....	11
3.3 The Data Layer: RDS Database Tier	11
3.3.1 Database Isolation & Security	11
3.3.2 High Availability & Performance	12
3.3.3 Automated Integration	12
4. Secret Management: HashiCorp Vault.....	12
5. Terraform Project Structure (File Map)	13
PART 2	14
6. Configuration Management (Ansible)	14
6.1 Dynamic Inventory Management	14
6.2 Playbook Structure & Role Delegation.....	14
6.3 The Automation Workflow	16
PART 3:.....	17
7. The Application Environment.....	17
8. Conclusion	18

1. Introduction

This project demonstrates the design and deployment of a highly available, secure, **3-Tier Web Architecture** on Amazon Web Services (AWS). By leveraging the power of **Infrastructure as Code (IaC)** with Terraform and **Configuration Management** with Ansible, we have transformed a complex manual setup into a fully automated, repeatable pipeline.

2. Overview

Problem vs Solution: Traditional manual infrastructure deployment is prone to human error, difficult to scale, and lacks consistency. This project solves these challenges by:

- **Automation:** Using Terraform to provision 100% of the hardware (VPC, RDS, Load Balancers).
- **Consistency:** Using Ansible to ensure every server is configured identically.
- **Security:** Implementing a "Zero-Trust" networking model where the database is completely isolated from the public internet.
- **Secret Management:** Utilizing **HashiCorp Vault** to eliminate hardcoded credentials, ensuring that AWS access keys and sensitive data are stored and retrieved through a secure, encrypted centralized authority.

Core Technology Stack

- **Cloud Provider:** AWS (VPC, EC2, ASG, ALB, RDS).
- **Infrastructure as Code:** **Terraform**, utilizing a modular structure for scalability.
- **Configuration Management:** **Ansible**, using roles and dynamic inventories to manage scaling instances.
- **Application Environment:** **Node.js** (Backend) and **Nginx/HTML/JS** (Frontend).
- **Database:** **PostgreSQL** managed via AWS RDS.
- **Secret Management:** **HashiCorp Vault**, for secure storage and dynamic retrieval of AWS credentials.

PART 1

3. System Architecture design (By Terraform)

The architecture is divided into three distinct logical layers, distributed across multiple **Availability Zones (AZs)** to ensure that the application remains online even if one AWS data center experiences an outage.

A **Networking Layer** that provides a secure, multi-AZ foundation; a **Compute Layer** that manages frontend Nginx and backend Node.js instances via Auto Scaling Groups and Application Load Balancers; and a **Data Layer** utilizing an isolated AWS RDS PostgreSQL instance. This modular approach ensures that each component—networking, compute, and storage—is decoupled, allowing for maximum security, scalability, and simplified management.

3.1. Networking Layer (The Infrastructure Foundation)

The networking layer is the bedrock of the entire 3-tier architecture. It defines the boundaries, access rules, and communication paths for every other resource. In this project, we designed a custom **Virtual Private Cloud (VPC)** that prioritizes high availability and strict traffic isolation.

3.1.1 Virtual Private Cloud (VPC) & Subnet Architecture

The network is built on a 10.0.0.0/16 CIDR block, providing ample IP space for current and future resources. To ensure the application remains online during an AWS data center failure, we deployed across two **Availability Zones (AZs)**.

```
## Networking layer
module "network" {
  source = "../modules/vpc"

  vpc_cidr      = "10.0.0.0/16"
  public_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  private_subnets = ["10.0.3.0/24", "10.0.4.0/24"]
  azs           = ["us-east-1a", "us-east-1b"]
}
```

The subnet design follows a strict isolation strategy:

Subnets (7) [Info](#)

Find subnets by attribute or tag

☐	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
☐	privateSubnet-1	subnet-031c0be6931d0d558	Available	vpc-0a6422a79493529fc main...	Off	10.0.4.0/24
☐	-	subnet-03c4f81e9e9df42f2	Available	vpc-0b97d86366dc374f8	Off	172.31.0.0/20
☐	privateSubnet-0	subnet-04656793c6db64640	Available	vpc-0a6422a79493529fc main...	Off	10.0.3.0/24
☐	-	subnet-0d2b14163a0ecb9f7	Available	vpc-0b97d86366dc374f8	Off	172.31.16.0/2
☐	publicSubnet-0	subnet-0f6e2f9e992a4bb5	Available	vpc-0a6422a79493529fc main...	Off	10.0.1.0/24
☐	-	subnet-072b5c18a3f54e7f0	Available	vpc-0b97d86366dc374f8	Off	172.31.32.0/2
☐	publicSubnet-1	subnet-0adebbau641f092dc	Available	vpc-0a6422a79493529fc main...	Off	10.0.2.0/24

- **2 Public Subnets:** These are the "External Zones." They are associated with an **Internet Gateway (IGW)**, allowing them to host the Public Application Load Balancer and the Bastion Host.

Internet gateways (2) [Info](#)

Find internet gateways by attribute or tag

☐	Name	Internet gateway ID	State	VPC ID	Owner
☐	-	igw-009a37e5e7a785c5a	Attached	vpc-0b97d86366dc374f8	663702377195
☐	mainIGW	igw-05d00df0ed692e457	Attached	vpc-0a6422a79493529fc mainVPC	663702377195

- **2 Private Subnets:** These are the "Internal Zones." They have no direct route to the internet, providing a secure environment for Backend EC2 instances, the frontend EC2 instances and the RDS. Communication to the internet (for software updates) is handled safely through a **NAT Gateway**.

NAT gateways (4) [Info](#)

Find NAT gateways by attribute or tag

☐	Name	NAT gateway ID	Connectivity...
☐	NatGateway-1	nat-0bad5c7ac213b72d2	Public
☐	NatGateway-0	nat-05727ee62181f2926	Public

3.1.2 Connectivity & Routing Logic

Traffic flow is controlled by custom route tables that define exactly how data moves between tiers:

Public Route Table: Maps traffic from the public subnets to the **Internet Gateway**.

Private Route Table: Maps outbound traffic from the private subnets to the **NAT Gateway**, ensuring that internal servers can reach the internet but the internet cannot reach them directly.

Route tables (1/5) Info

Last updated 7 minutes ago

Actions

Create route table

☒	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☒	Public-rtb	rtb-0f384406b838d6bb3	2 subnets	-	No	vpc-0a6422a79493529fc ma
☐	-	rtb-042e0df45d60620af	-	-	Yes	vpc-0a6422a79493529fc ma
☐	-	rtb-062cf70c1449c344e	-	-	Yes	vpc-0b97d86366dc374f8
☐	Private-rtb-0	rtb-0145a1b36b8bddd41	subnet-04656793c6db64...	-	No	vpc-0a6422a79493529fc ma

rtb-0f384406b838d6bb3 / Public-rtb

Details

Routes

Subnet associations

Edge associations

Route propagation

Tags

Routes (2)

Both

Edit routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-05d00df0ed692e457	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

Route tables (1/5) Info

Last updated 11 minutes ago

Actions

Create route table

☒	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☐	Public-rtb	rtb-0f384406b838d6bb3	2 subnets	-	No	vpc-0a6422a79493529fc ma
☐	-	rtb-042e0df45d60620af	-	-	Yes	vpc-0a6422a79493529fc ma
☐	-	rtb-062cf70c1449c344e	-	-	Yes	vpc-0b97d86366dc374f8
☒	Private-rtb-0	rtb-0145a1b36b8bddd41	subnet-04656793c6db64...	-	No	vpc-0a6422a79493529fc ma

rtb-0145a1b36b8bddd41 / Private-rtb-0

Details

Routes

Subnet associations

Edge associations

Route propagation

Tags

Routes (2)

Both

Edit routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	nat-08480ee22b8c55a45	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

3.1.3 Load Balancing Architecture (The Traffic Gateways)

In this 3-tier design, we deployed two separate Application Load Balancers to manage traffic flow and ensure high availability. By separating public and internal traffic, we create a clear boundary between the user and the application's private logic.

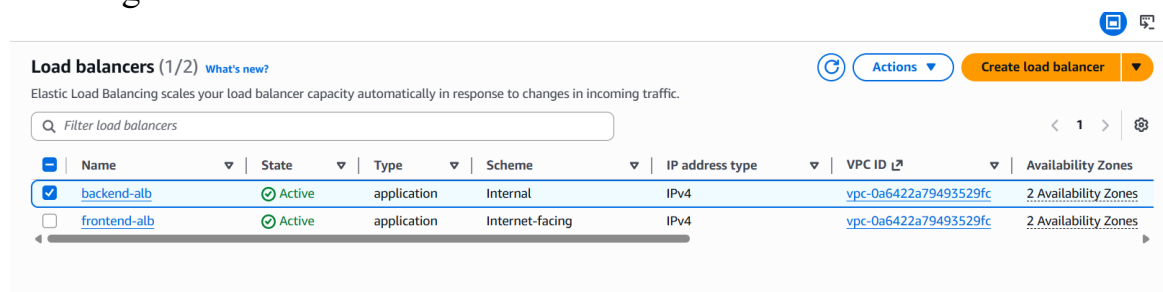
1. Public Frontend ALB (External)

- **Placement:** Deployed across the **Public Subnets**.
- **Function:** This is the entry point for all end-users. It receives incoming HTTP traffic from the internet and distributes it across the Frontend EC2 instances.

- **Why we did this:** It provides a single stable DNS name for the website. If one Frontend instance fails, the ALB automatically detects it and redirects traffic to a healthy instance, preventing any downtime for the user.

2. Internal Backend ALB (Private)

- **Placement:** Deployed across the **Private App Subnets**.
- **Function:** This ALB is **Internal-only**. It acts as the bridge between the Frontend and the Backend. When the Frontend needs to save or retrieve data, it sends an API request to this Internal ALB.
- **Why we did this:** By using an internal balancer, we keep the Backend instances completely hidden. The Frontend doesn't need to know the specific IP addresses of the Backend servers; it only needs to know the Internal ALB's DNS name. This allows the Backend to scale up or down without breaking the connection.



	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones
☒	backend-alb	Active	application	Internal	IPv4	vpc-0a6422a79493529fc	2 Availability Zones
☐	frontend-alb	Active	application	Internet-facing	IPv4	vpc-0a6422a79493529fc	2 Availability Zones

3. Health Checks & Reliability

Both ALBs utilize **Target Groups** with active health checks.

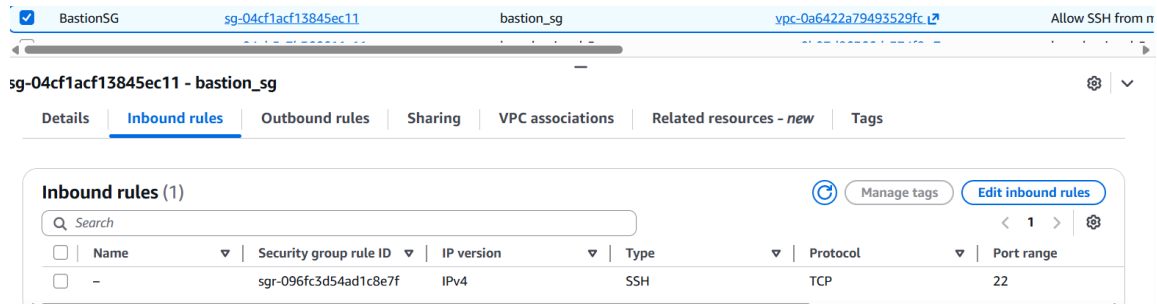
- **The Logic:** Every 30 seconds, the ALB "pings" a specific path on the servers.
- **The Benefit:** If an Ansible update or a code error causes a server to crash, the ALB stops sending traffic to that specific server immediately. This ensures that the user only ever interacts with a fully functional part of the system.

3.1.4. Security Group Chaining

Security Group Chaining is the strategic practice of using the Security Group ID of one tier as the "Source" for the next. This ensures that traffic is not just filtered by port and IP, but by the **identity** of the resource sending the data. This creates a virtual "tunnel" where data must follow a strictly defined path.

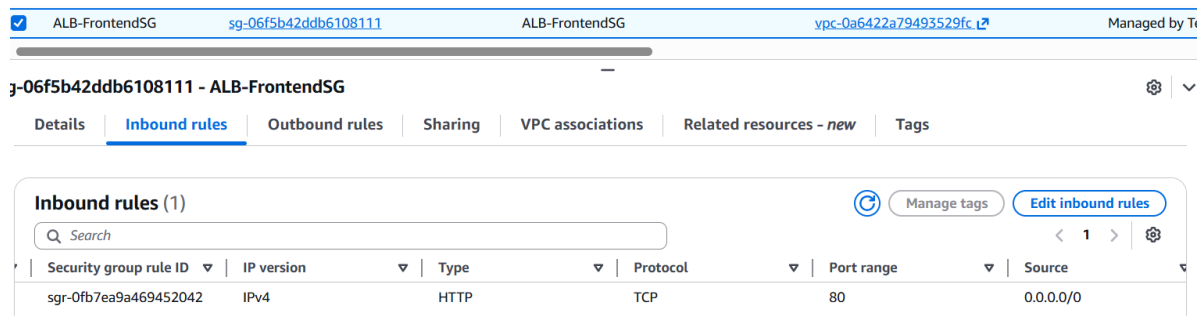
1. Bastion Host Security Group

- **Access Rule:** Allows SSH (Port 22) only from a specific trusted IP (the administrator's IP).
- **Why:** Since the Bastion is the only "front door" to the private network for management, we keep it locked to everyone except the authorized engineer. It acts as the secure jump point for all internal maintenance.



2. Frontend ALB Security Group

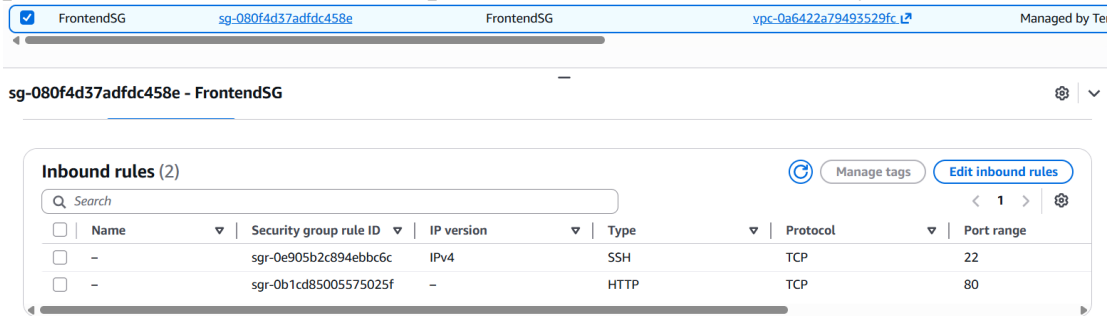
- **Access Rule:** Allows HTTP (Port 80) from 0.0.0.0/0 (the entire internet).
- **Why:** As the public face of the application, the Frontend Load Balancer must be reachable by any user trying to access the website. It is the only component in the entire stack (besides the Bastion) that "listens" to the open internet.



3. Frontend EC2 Security Group

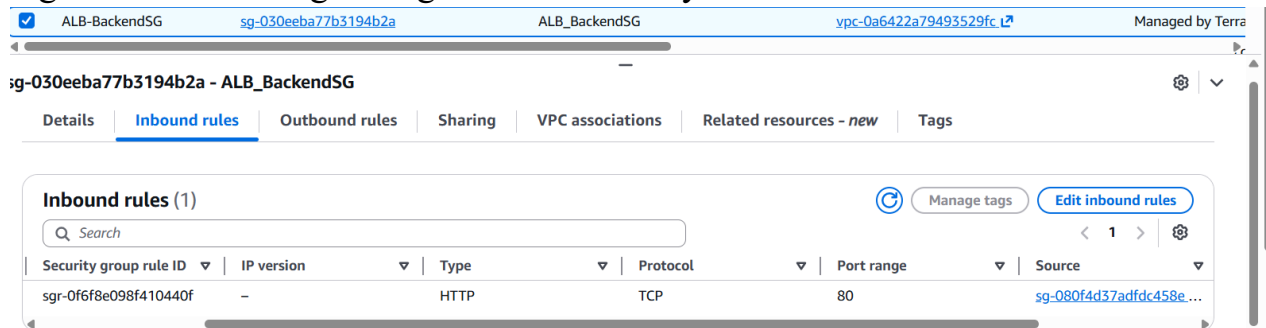
- **Access Rule:** Allows Port 80 strictly from the **Frontend ALB Security Group ID**.
- **Why:** We do not want users to bypass the Load Balancer and hit the servers directly. By chaining the SG, we ensure that the Frontend instances only

process traffic that has been pre-filtered and distributed by the ALB.



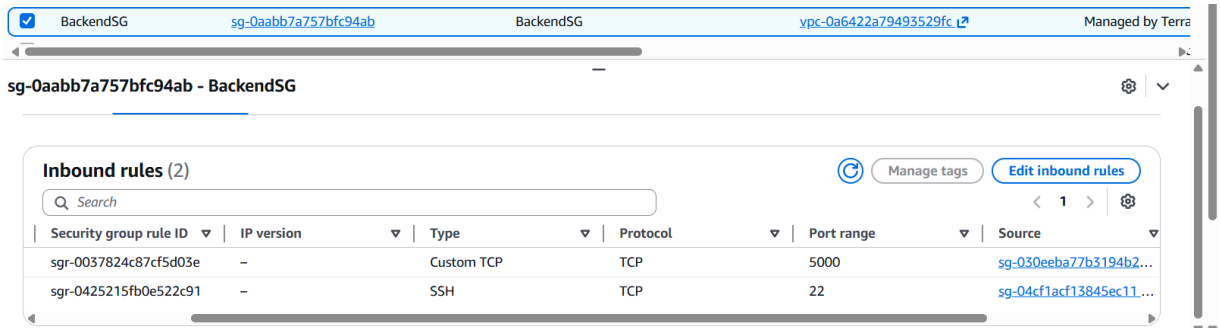
4. Backend ALB Security Group

- **Access Rule:** Allows HTTP traffic only from the **Frontend EC2 Security Group ID**.
- **Why:** The Backend ALB is internal. It serves as a private bridge. By only allowing the Frontend instances to talk to it, we ensure that even if someone managed to get into the VPC, they couldn't send requests to the backend logic without coming through the frontend layer first.



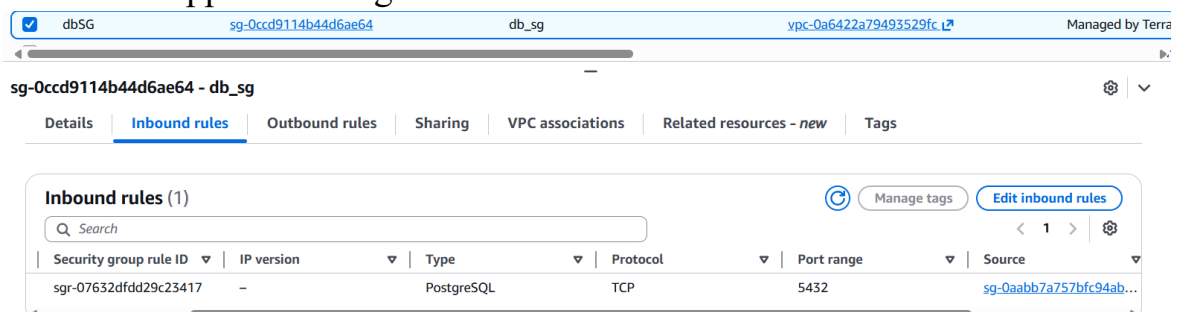
5. Backend EC2 Security Group

- **Access Rule:** Allows application traffic from the **Internal ALB Security Group ID** and SSH from the **Bastion Security Group ID**.
- **Why:** This isolates the business logic. The backend only performs work when requested by the Internal ALB. The SSH rule allows the administrator to manage the server via the Bastion without exposing Port 22 to the public.



6. RDS Database Security Group

- **Access Rule:** Allows PostgreSQL traffic (Port 5432) strictly from the **Backend EC2 Security Group ID**.
- **Why:** The database contains the most sensitive data (patient records). By chaining it to the Backend SG, we guarantee that no other resource—not even the Bastion or the Frontend—can reach the database directly. Only the authorized application logic can read or write data.



3.2. The Compute Layer: Application & Logic Tier

The compute layer is where the actual application code resides. In this project, we moved away from standalone servers to a **highly available, auto-scaling architecture**. This ensures that the application can handle varying levels of traffic while automatically recovering from hardware failures.

3.2.1 Dual-Tier EC2 Architecture

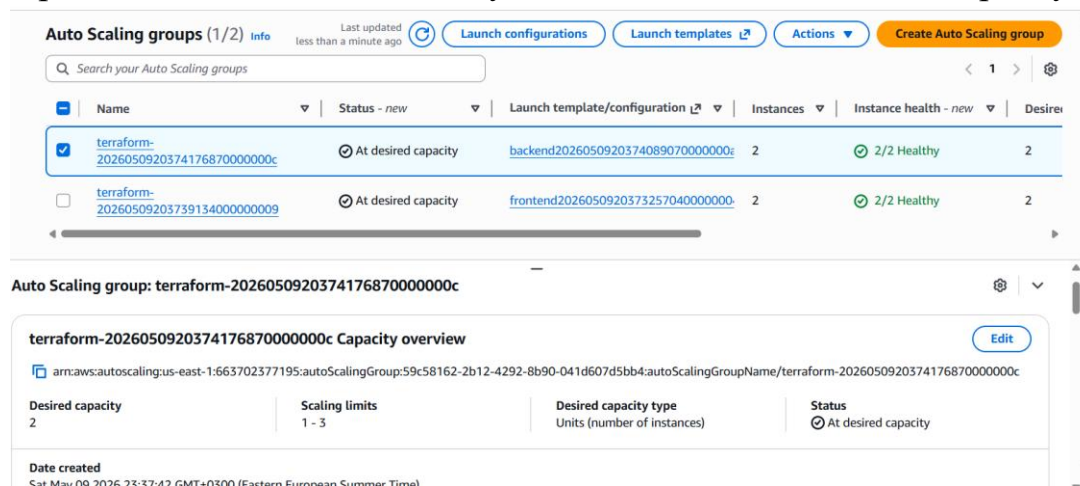
The compute layer is split into two specialized groups of EC2 instances, both residing in the **Private App Subnets** for maximum security.

- **Frontend Tier (Nginx & Web Assets):** These instances act as the web servers. They run Nginx to serve the static frontend files (HTML/JS/CSS) and proxy requests to the backend.
- **Backend Tier (Node.js API):** These instances run the business logic. They handle API requests, process data, and communicate with the PostgreSQL database.

3.2.2 High Availability with Auto Scaling Groups (ASG)

To ensure the system is "self-healing," we implemented **Auto Scaling Groups** for both the frontend and backend tiers.

- **Launch Templates:** We defined a "gold image" (AMI), instance type (t3.micro), and user-data scripts in a Launch Template. This ensures every new instance is a perfect clone of the original.
- **Desired Capacity:** We configured the ASG to maintain a minimum of **two instances** per tier, spread across different Availability Zones.
- **Auto-Recovery:** If an instance becomes unhealthy or a whole AWS zone goes down, the ASG automatically detects the failure and launches a replacement instance in a healthy zone to maintain the desired capacity.



3.2.3 Dynamic Scaling & Resource Management

By using ASGs in conjunction with the Application Load Balancers discussed in the networking section, the compute layer gains several advantages:

- **Zero Downtime Updates:** We can roll out new versions of the code by updating the Launch Template and letting the ASG replace instances one by one.
- **Efficiency:** The system only runs the number of instances required, optimizing costs while ensuring performance.

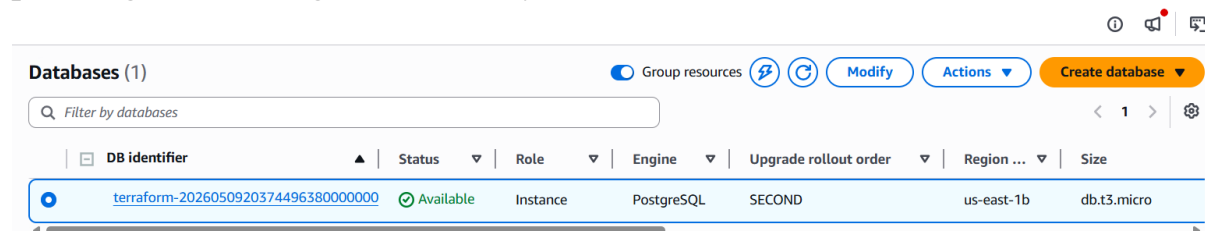
3.2.4 Integration with Ansible

The compute layer was designed to be "dumb" hardware that is "made smart" by Ansible. Terraform creates the "empty" EC2 instances, and then Ansible connects to them via the **Bastion Host** to:

1. Install necessary software (Node.js, Nginx, Git).
2. Clone the application repositories.
3. Inject environment variables (like the RDS endpoint generated by Terraform).
4. Start the services.

3.3 The Data Layer: RDS Database Tier

The Data Layer is the most critical part of the architecture, as it houses the persistent storage for the application. For this project, we chose **Amazon RDS for PostgreSQL** to leverage a managed database environment that handles backups, patching, and scaling automatically.



The screenshot shows the AWS Management Console interface for the 'Databases' section. At the top, there are buttons for 'Group resources', 'Modify', 'Actions', and 'Create database'. Below these is a search bar labeled 'Filter by databases'. A table lists the database instances with columns: DB identifier, Status, Role, Engine, Upgrade rollout order, Region, and Size. One instance is listed: 'terraform-202605092037449638000000' with a status of 'Available', role of 'Instance', engine of 'PostgreSQL', and size of 'db.t3.micro'.

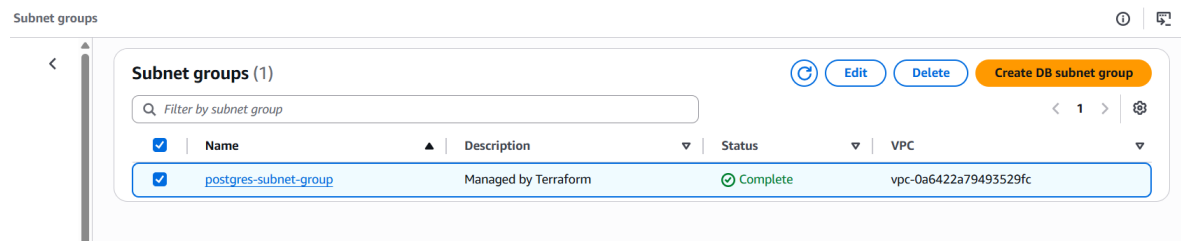
DB identifier	Status	Role	Engine	Upgrade rollout order	Region	Size
terraform-202605092037449638000000	Available	Instance	PostgreSQL	SECOND	us-east-1b	db.t3.micro

3.3.1 Database Isolation & Security

Security was the primary driver for the design of this layer. The database is shielded by multiple layers of protection:

- **Private Subnet Placement:** The RDS instance is deployed into a **DB Subnet Group** consisting of the two most restricted private subnets. It has

no Public IP and is physically unreachable from the internet.



- **Network ACLs & Security Groups:** As discussed in the "Chaining" section, the database only accepts traffic on **Port 5432** from the Backend EC2 Security Group.

3.3.2 High Availability & Performance

To ensure the data is always available and safe from corruption, we implemented the following:

- **Multi-AZ Readiness:** While we used a Single-AZ deployment for this phase, the subnet group is configured across two zones to allow for an easy upgrade to a "Multi-AZ" failover configuration in the future.
- **Storage Autoscaling:** We enabled storage autoscaling to ensure that as patient records and logs grow, the database volume expands without manual intervention or downtime.

3.3.3 Automated Integration

One of the most complex parts of a 3-tier setup is telling the App how to find the Database. We solved this through **Terraform Outputs**:

1. Terraform provisions the RDS instance and generates a unique **Endpoint URL**.
2. This endpoint is automatically captured and passed to the Ansible configuration.
3. Ansible then injects this URL into the Backend application's environment variables.

4. Secret Management: HashiCorp Vault

To adhere to enterprise security standards, we integrated **HashiCorp Vault** as our centralized secret engine. Instead of hardcoding sensitive credentials or

storing them in plain-text `tfvars` files, the infrastructure pulls data dynamically during the `apply` phase.

Credential Decoupling: AWS Access Keys and RDS administrative passwords are stored in encrypted Vault kv-engines.

Authentication: Terraform authenticates to Vault using a secure token, retrieving only the secrets necessary for the current environment.

Zero-Persistence: Sensitive data exists only in memory during the deployment process, significantly reducing the risk of credential leakage in version control systems (Git).

5. Terraform Project Structure (File Map)

To wrap up the **Part 1: Terraform Infrastructure** section of your report, here is the complete file map showing how the project is organized. This structure demonstrates a high level of professional modularity.

3-Tier-Project/

```
|— modules/
|   |— vpc/    # VPC, Subnets, Gateways, and Routing
|   |— alb/    # Public and Internal Load Balancers
|   |— ec2/    # Launch Templates and Auto Scaling Groups
|   |— rds/    # PostgreSQL Instance and Subnet Groups
|— main.tf    # Root configuration linking all modules
|— providers.tf # AWS and Vault provider definitions
|— variables.tf # Global input variables
|— outputs.tf  # Dynamic outputs (ALB DNS, RDS Endpoints)
```


PART 2

6. Configuration Management (Ansible)

Once Terraform has provisioned the "hardware" (the EC2 instances, network, and database), Ansible is used to transform those raw virtual machines into functional web and application servers. This process is fully automated, ensuring that every instance—regardless of when it was launched—is configured identically.

6.1 Dynamic Inventory Management

Dynamic Inventory Using AWS EC2 Plugin

- This file uses the **amazon.aws.aws_ec2** dynamic inventory plugin to automatically discover EC2 instances from AWS.
- The inventory is configured to search in the us-east-1 AWS region.
- It uses the AWS CLI/profile credentials named **ansible-proj** to connect to the AWS account.
- Hostnames are generated using the EC2 instance ID, which makes each host uniquely identifiable.
- Instances are automatically grouped based on their Name tag:
 - Instances with **"frontend"** in their Name tag are added to the frontend group.
 - Instances with **"backend"** in their Name tag are added to the backend group.
- The **ansible_host** value is set dynamically:
 - It uses the public IP address if available.
 - If no public IP exists, it falls back to the private IP address.
- The SSH user is configured as **ec2-user**, which is the default user for Amazon Linux EC2 instances.
- This setup removes the need to manually update inventory files when EC2 instances are created, replaced, or terminated.

6.2 Playbook Structure & Role Delegation

The deployment is orchestrated through a master playbook (playbook.yaml) that delegates tasks to specific roles. This modularity allows us to reuse code and maintain a clean separation of concerns.

1. Basic Setup Role

Applied to **all hosts**, this role ensures the base environment is ready.

Tasks:

- Installs **Docker** on all EC2 instances using the yum package manager.
- Updates the package cache before installation.
- Creates the Docker CLI plugins directory at:

/usr/local/lib/docker/cli-plugins

- Downloads and installs the **Docker Compose v2 plugin** from GitHub.
- Sets executable permissions for the Docker Compose plugin.
- Notifies the Docker handler to start Docker after installation.

2. Backend Role

Applied strictly to the **backend host group**.

Tasks:

- **Copy Backend Files:** Copies the backend application files from the Ansible server to the backend EC2 instance.
- **Install PostgreSQL Client:** Installs the PostgreSQL 15 client so the instance can connect to the RDS database.
- **Initialize RDS Schema:** Runs the dbssl.sql file against the RDS database to create or prepare the backend database schema.
- **Copy Docker Compose File:** Generates and copies the backend docker-compose.yml file from a Jinja2 template.
- **Run Backend Containers:** Runs Docker Compose, builds the required images, and starts the backend containers.

3. Frontend Role

Applied strictly to the **frontend host group**.

Tasks:

- **Copy Frontend Files:** Copies the frontend application files from the Ansible server to the frontend EC2 instance.
- **Add Nginx Configuration:** Generates and adds an nginx.conf file to proxy backend API requests to the internal backend ALB.
- **Copy Docker Compose File:** Copies the frontend docker-compose.yml file to the project directory on the instance.
- **Run Frontend Containers:** Runs Docker Compose, builds the required images, and starts the frontend container.

6.3 Ansible Project Structure

To wrap up **Part 2: Ansible Configuration Management** section of the report, here is the complete file map showing how the Ansible project is organized. This structure demonstrates a clean role-based approach for automating the deployment process.

three-tier-proj-ansible/

```
|— group_vars
|— inventory.aws_ec2.yml
|— playbook.yml
└─ roles
    └─ backend
    └─ basic-setup
    └─ frontend
```

6.4 The Automation Workflow

The integration between Terraform and Ansible creates a seamless deployment pipeline:

Terraform creates the instances and tags them.

Ansible executes the dynamic inventory to "see" the new instances.

The Playbook runs through the **Bastion Host** (acting as an SSH Proxy) to securely configure the private instances without exposing them to the internet.

PART 3

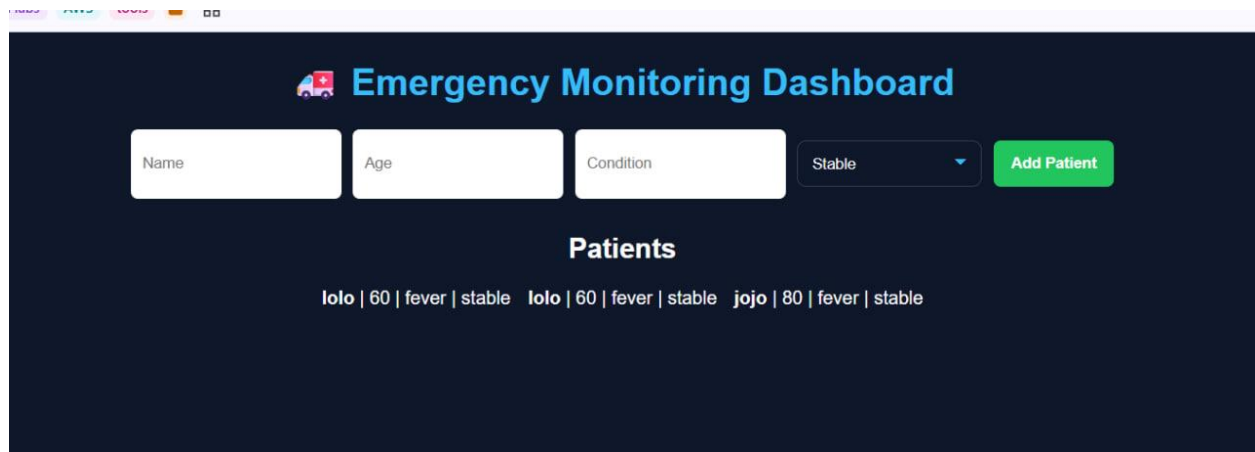
7. The Application Environment

The final layer of the project is the application itself, designed as a decoupled full-stack system.

Frontend: A lightweight web interface built with HTML/JS and served by Nginx.

Backend: A Node.js API that processes business logic.

Database: A PostgreSQL instance managed by RDS, which stores all persistent application data.



8. Conclusion

The successful deployment of this 3-tier architecture represents a shift from manual, error-prone infrastructure management to a modern, **Software-Defined Infrastructure** approach. By integrating **HashiCorp Vault** for secure credential handling, **Terraform** for modular provisioning, and **Ansible** for dynamic configuration, we have created a system that is not only highly available but also adheres to the highest security standards.

Key Outcomes:

- **Operational Efficiency:** The time required to provision and configure the entire environment has been reduced from hours to minutes through a single-command deployment pipeline.
- **Zero-Trust Security:** By utilizing Security Group chaining and placing all application and database resources in private subnets, we have ensured that sensitive data is shielded from public internet threats.
- **Self-Healing Resilience:** Through the use of Auto Scaling Groups and multi-AZ networking, the system is designed to automatically recover from instance or zone failures without manual intervention.